Programming Assignment 1 (PA1)
Out: February 2nd, 2026, Monday -- DUE: February 18th, 2026, Wednesday, 11:59pm
EC327 Introduction to Software Engineering – Spring 2026

Total: 100 points

- *You may use any development environment you wish, as long as it is ANSI C++ compatible. Please make sure your code compiles and runs properly under the Linux environment on the PHO307/305 (or eng-grid) machines before submitting.*
- *PAs may be submitted up to a week late at the cost of a **30% fixed penalty** (e.g., submitting a day late and a week late is equivalent). It is in your best interest to complete as many PA questions as possible before the deadline. If you have missing questions in your original submission, you may complete and submit the missing solutions during the following week. Any submissions after the deadline will be subject to the 30% penalty. No credit will be given to solutions submitted after the 1-week late submission period following the deadline.*
- *Please make sure to follow the assignment submission guidelines for **GitHub**. These will be posted in a separate document.*

Please write C++ code for solving the following problems related to data types, operations, if statements, simple loops, and basic functions (Chapters 1-2-3-4-5-6 of the textbook by Horstmann).

Q1: *Shortest Distance from a Point to a Line* [20 points]
Write a program that reads three x, y coordinates from the console (STDIN). The first two coordinates will be used to compute an equation of a straight line passing through each coordinate. The program should then calculate the shortest distance from the third coordinate to the line and print the result to the console (STDOUT). You may use the internet to learn the equations but NOT get code. **We will be checking for this.**

Text in blue in the examples demonstrates the user inputs entered via the keyboard.

Sample run:
Enter three x,y coordinates:
1 2 2 3 2 4<enter>
The shortest distance for (2,4) to the line composed of (1,2) and (2,3) is ?**. [Program Exits]

**You should display at least two fractional digits. You can display more of the fractional part if you like.*
***Your program needs to replace ? with the answer.* ☺

Q2. *Tip and Tax Calculator* [15 points]
Write a program that calculates the tip amount, tax amount, and total costs on a restaurant bill. Your program should allow for three tip levels (10%, 15%, 20%, 25%) and determine if the tip should be calculated over just the original amount or the (original amount + tax). **If an incorrect code is entered (e.g. 6 for tip level or tax option) an error message should be shown and the user should be prompted again.**

Sample run:
Enter the original bill amount: 56.75 <enter>
Enter the tax rate %: 14.3 <enter>
Enter tip level (1=10%, 2=15%, 3=20%, 4=25%): 4<enter>
Tip on post-tax amount? (1=yes, 2=no): 2<enter>
The total bill pre-tip is: $64.87
The total tip is: $14.19
The total bill post-tip is: $79.06
[Program Exits]

**You should display at only two fractional digits. Do independent research to see how to do this.*

Q3. *Hamming Distance* [25 points]

In information theory, the **Hamming distance** between two sequences of equal length is the number of positions at which the corresponding symbols are different. Put another way, it measures the minimum number of *substitutions* required to change one string into the other, or the number of *errors* that transform one string into the other.

Examples:

The Hamming distance between:

- "**toned**" and "**roses**" is 3.
- **1011101** and **1001001** is 2.
- **2173896** and **2233796** is 3.

(Description above is from Wikipedia: http://en.wikipedia.org/wiki/Hamming_distance)

Write a program that prompts the user to enter two numbers between 0-100 (decimal) and computes the Hamming distance between the two numbers when the numbers are represented in base-3 (ternary format). The program then displays the Hamming distance on the screen. **Hint: You don't need to convert the numbers to 1's, 2's, and 0's to figure this out. Think about how to figure this out with arithmetic. Start out thinking how you would do this with base-10.**

Sample runs:
Enter two numbers between 0-100: <8> <enter>
<23> <enter>
Hamming distance between 8 and 23 when numbers are in ternary format is: 2 [Program Exits]*

Enter two numbers between 0-100: <27> <enter>
<30> <enter>
Hamming distance between 27 and 30 when numbers are in ternary format is 1. [Program Exits]**

** 8 and 23 are represented as 022 and 212, respectively, in ternary form.*
***27 and 30 are represented as 1000 and 1010 in ternary form.*

Q4. *Fire, Water, Grass* [20 points]

Write a program to play "Fire, Water, Grass". In this game, you first prompt the user to choose "Fire, Water, or Grass". After this, the computer should randomly generate one of these three. Then you should display the choice of the computer and determine the winner (Fire beats Grass, Grass beats Water, Water beats Fire).

Sample run:
Choose Fire (0), Water(1), or Grass (2): *<1> <enter>*
Computer chooses: Fire
You win!
[Program Exits]

Choose Fire (0), Water(1), or Grass (2): *<2> <enter>*
Computer chooses: Fire
You lose!
[Program Exits]

Choose Fire (0), Water(1), or Grass (2): *<0> <enter>*
Computer chooses: Fire
You tie!
[Program Exits]

**Hint: http://www.cplusplus.com/reference/cstdlib/rand/*

Q5. *Cryptography* [20 points]
There are many ways to encrypt messages. One way is to write every word with the next lexigraphical letter such as; "Hello World" would be encrypted as "IFMMP XPSME". Write code that takes a letter (in lowercase or uppercase) as input and prints this encryption of it; which is the next letter (in a cyclic way i.e. for 'Z' it should print 'A'). The output should be in uppercase. If the input is not a letter (e.g., user entered a number instead of a letter), your code should print an error message.

Hint: You might find ASCII table useful to print the next letter or convert to uppercase. In the ASCII table (see Appendix B of your book), uppercase letters appear before lowercase letters. The offset between any uppercase letter and its corresponding lowercase letter is the same. You can compute this offset by:
int offset = 'a' – 'A'; *or by* int offset = 'A' – 'a';

Sample runs:
Enter a letter: n<enter>
CRYPTO: O [Program Exits]

Enter a letter: N<enter>
CRYPTO: O [Program Exits]

Enter a letter: Z<enter>
CRYPTO: A [Program Exits]

Enter a letter: 2<enter>
YOU DID NOT ENTER A LETTER [Program Exits]

IMPORTANT NOTE: YOUR CODE MAY GO THROUGH AN "AUTOGRADER" SO MAKE SURE YOUR CODE TAKES THE INPUTS IN THE SAME ORDER AS THE ABOVE EXAMPLE AND ITS OUTPUT IS EXACTLY "CRYPTO:**" OR "**YOU DID NOT ENTER A LETTER" **AS IN THE ABOVE EXAMPLE (CAPSLOCK AND SAME SENTENCE)**

Submission Details

Each program should be a single file. Please use the file names **Q1.cpp, Q2.cpp, etc.** for questions 1-5, respectively. Make sure to **comment** your code. **We will provide more information on how to submit this to GitHub** (where all PAs for this semester will be submitted). Do **NOT** submit your executable files (a.out or others). We only need the source code.